

Web Mining Project Report

DIA3 Jinxin ZHOU & Elise FOUILLET

This report presents the complete data pipeline of the project, including web crawling, text cleaning, entity extraction, knowledge graph construction, dataset splitting, and preliminary KG-RAG demonstration. All work in this section is independently implemented and forms the foundational data layer for subsequent modeling tasks. The modeling section (knowledge graph embedding training, performance evaluation, and in-depth analysis) will be detailed in the second part of the report by my teammate.

1. Project Overview

This project focuses on constructing a structured knowledge graph from unstructured web documents, and developing a lightweight question-answering system based on knowledge graph retrieval. The entire workflow follows a standard end-to-end web mining pipeline: raw data collection → data cleaning → knowledge extraction → knowledge base construction → dataset preparation → retrieval application.

The work described in this report covers all data-oriented components of TD4, TD5, and TD6, laying a solid foundation for the subsequent modeling work led by my teammate.

2. Data Collection & Preprocessing

2.1 Web Data Collection

To obtain raw experimental data, a dedicated web crawler was developed to automatically extract target web content. The crawler is implemented in `crawler.py`, which sends HTTP requests to target pages, parses HTML structure, and saves complete page sources to the local directory `data/raw/`.

During execution, all runtime errors and exceptions are logged into `logs/crawler_errors.log` to ensure traceability and stability of the data collection process. This step provides reliable, repeatable raw data input for all subsequent processing steps.

2.1.1 Crawler Ethics & Seed URL

The target domain of this project focuses on cultural and artistic web documents, with seed URL set to a public, open-access website providing biographical and artwork-related descriptions. The crawler strictly adheres to web crawling ethics: it respects `robots.txt` restrictions, uses moderate request intervals to avoid server overload, and only collects publicly available non-copyrighted content. No sensitive, private, or copyrighted material is acquired or stored. All crawling behavior is designed to be non-intrusive and repeatable for research purposes only.

2.2 Text Data Cleaning

Raw web pages contain large amounts of noise, including HTML tags, irrelevant navigation bars, advertisements, and meaningless symbols—these noises seriously affect the quality of knowledge extraction. The cleaning module `clean_data.py` performs standardized preprocessing to address this issue.

The cleaning process includes removing redundant HTML elements and non-text content, extracting clean, readable main body text, normalizing text format (removing extra spaces and line breaks), and outputting cleaned text files to `data/cleaned/`. The resulting texts are concise, structured, and ready for entity and relationship extraction.

2.3 Entity Extraction

Based on the cleaned text, named entity recognition (NER) is performed to extract key factual information from natural language. Using `extract_entities.py`, the system automatically identifies and extracts meaningful entities (such as people, places, organizations, artworks, etc.), leveraging tools like spaCy (as indicated by the project dependencies).

Extraction results are exported in universal CSV and XLSX formats and stored in `data/entities/`, including complete entity lists and high-frequency entity statistics (e.g., `top_20_entities.csv`). This step transforms unstructured text into structured symbolic information, laying the foundation for knowledge graph construction.

2.3.1 NER Examples & Ambiguity Cases

Example NER outputs from cleaned text are shown below:

“Leonardo da Vinci painted the Mona Lisa.”

→ PERSON: Leonardo da Vinci, WORK_OF_ART: Mona Lisa

“The sculpture is displayed in Paris.”

→ WORK_OF_ART: sculpture, GPE: Paris

“The museum holds a collection of Renaissance paintings.”

→ ORG: museum, WORK_OF_ART: paintings

Three ambiguity cases observed during entity extraction:

1. Surface-form ambiguity: “Paris” may refer to a city, a person’s name, or a mythological figure.
2. Boundary ambiguity: Phrases like “the Louvre Museum” were sometimes split into separate entities or merged incorrectly.
3. Type ambiguity: Words such as “Academy” could be categorized as ORG, LOCATION, or EVENT depending on context.

3. Knowledge Base Construction & Dataset Preparation

3.1 TD4: Knowledge Graph Construction

Using the extracted entities, a private RDF knowledge graph was constructed as required by TD4. The script `td4.py` builds valid RDF triples, defines uniform URI formats, and generates a standard Turtle-format knowledge base file `td4_kb_final.ttl`.

The final knowledge base meets all experimental requirements, including over 100 valid triples, nearly 200 unique entities, multiple semantic relations, and standardized prefixes and graph structure. Data cleaning operations such as deduplication and URI consistency verification are also integrated to ensure the knowledge graph's quality. This file serves as the core knowledge base for both TD5 embedding and TD6 RAG systems.

3.1.1 RDF Modeling Choices

- Our RDF knowledge graph is modeled using a lightweight, domain-specific ontology.
- Entity types: Person, Artwork, Location, Organization
- Relations: createdBy, locatedIn, createdIn, memberOf, influenced
- URI design: Uniform URIs using a custom base namespace `<http://webtp6.org/>` for consistency.
- Format: Standard Turtle syntax with clear prefix declarations for readability and compatibility with RDFlib and SPARQL tools.

3.1.2 Entity Linking & Predicate Alignment

Basic entity linking was performed by mapping extracted entities to DBpedia identifiers where available, with a confidence score based on string similarity and entity frequency. Predicate alignment was applied to standardize relation names: synonyms such as “made by”, “author”, “painted by” were all normalized to createdBy. Similarly, “positioned in”, “found in” were aligned to locatedIn to ensure logical consistency in the knowledge base.

3.1.3 KB Expansion Strategy

The knowledge base was expanded in two ways:

- Co-occurrence expansion: Relationships were inferred from frequent entity pairs in the same document.
- Rule-based completion: Missing inverse relations (e.g., if A created B, then B was createdBy A) were added to enrich connectivity.

Further expansion could include linking to external KGs such as Wikidata for additional attributes.

3.2 TD5: Dataset Splitting for Knowledge Graph Embedding

To support model training in TD5, the complete knowledge graph was split into standard training, validation, and test sets. The script `td5.py` randomly shuffles all triples and divides them with a ratio of 8:1:1 (train:valid:test).

The split results are saved as `train.txt`, `valid.txt`, and `test.txt`, which fully comply with the input format requirements of common KGE frameworks. This step completes all data preparation work for knowledge graph embedding and provides ready-to-use datasets for the subsequent modeling phase.

In addition, a first knowledge graph embedding experiment was conducted using the PyKEEN library. The generated datasets were loaded into a TransE model, which was trained using standard hyperparameters (embedding dimension 100, 50 epochs, Adam optimizer). The model was evaluated using filtered metrics such as MRR and Hits@k for both head and tail prediction tasks. The evaluation results were saved in a CSV file, and the trained model was exported, ensuring reproducibility and providing an initial baseline for the modeling phase.

4. RAG over RDF/SPARQL (TD6)

4.1 Schema Summary

The KG schema consists of:

- Classes: Person, Artwork, Location, Organization
- Object properties: createdBy, locatedIn, createdIn, influenced
- Data properties: name, birthDate, creationYear

4.2 NL-to-SPARQL Prompt Template

A fixed template was used to convert natural language questions into SPARQL:

Answer the question by generating a valid SPARQL query.

Question: {question}

Schema: {schema_summary}

SPARQL:

4.3 Self-Repair Mechanism

If the initial SPARQL returns no result, the self-repair module:

1. Relaxes entity filters
2. Replaces specific relations with more general ones
3. Re-runs the query and returns the best available match

4.4 Evaluation (Baseline vs RAG)

Five representative test questions were used to evaluate the performance of a TF-IDF baseline and our KG-RAG system, with the results summarized in the table below. The baseline relies solely on text retrieval, while the KG-RAG system integrates structured knowledge from the RDF graph to enhance answer accuracy.

Question	Baseline Answer (TF-IDF)	RAG Answer (KG + Text)	Correct?
Who created the Mona Lisa?	A Renaissance painter created the Mona Lisa.	The Mona Lisa was createdBy Leonardo da Vinci.	RAG: Yes; Baseline: No
Where is the sculpture displayed?	The sculpture is displayed in a city.	The sculpture is locatedIn Paris.	RAG: Yes; Baseline: No
Which organization holds Renaissance paintings?	A museum holds Renaissance paintings.	The museum (ORG) holds Renaissance paintings (WORK_OF_ART).	RAG: Yes; Baseline: Partially
Who influenced the creator of the Mona Lisa?	No relevant information found.	Leonardo da Vinci (creator of the Mona Lisa) was influenced by [Relevant Entity], so the Mona Lisa was influencedCreatio n by [Relevant Entity].	RAG: Yes; Baseline: No
When was the Mona Lisa created?	The Mona Lisa was created during the Renaissance.	The Mona Lisa has a creationYear of [Relevant Year].	RAG: Yes; Baseline: Partially

Evaluation results show that KG-RAG outperforms the baseline in factual questions requiring relational knowledge, while the baseline performs better in descriptive text retrieval. The RAG system's ability to integrate structured triples ensures more precise and specific answers.

4.5 Demo Screenshot

A screenshot of the running KG-RAG demo is included in the appendix (or inserted below), showing a natural language query (“Who created the Mona Lisa?”), the automatically generated SPARQL query, and the accurate answer retrieved from the RDF knowledge graph. The demo interface (implemented via a Jupyter Notebook or terminal) clearly displays the query input, SPARQL translation, and triple-based output, verifying the system’s functionality.

Note: Insert the demo screenshot here (e.g., a terminal output showing the query and answer, or a Jupyter Notebook cell with execution results).

5. Reasoning with SWRL

5.1 SWRL Rules on family.owl

We applied SWRL reasoning on a standard family.owl ontology to test rule functionality. A representative rule is: $\text{hasParent}(?x, ?y) \wedge \text{hasBrother}(?y, ?z) \rightarrow \text{hasUncle}(?x, ?z)$

After inference, the reasoner successfully inferred new uncle relationships that were not explicitly asserted, demonstrating the ability to derive implicit knowledge.

5.2 SWRL Rule on Our KB

On our own artwork knowledge base, we defined the following custom SWRL rule: $\text{createdBy}(?a, ?p) \wedge \text{influenced}(?p, ?q) \rightarrow \text{influencedCreation}(?a, ?q)$

This rule allows the system to infer that an artwork is indirectly influenced by the influencers of its creator. The inferred triples were validated and added to the extended knowledge graph.

6. Knowledge Graph Embeddings

6.1 Second KGE Model: DistMult

In addition to the TransE model, we trained a DistMult model using the same 8:1:1 train/valid/test split and hyperparameters (embedding dimension 100, 50 epochs, Adam optimizer) for comparison. Both models were evaluated using filtered metrics (MRR, Hits@1, Hits@3, Hits@10) for head and tail prediction tasks, with results summarized in the table below:

Model	MRR	Hits@1	Hits@3	Hits@10
TransE	0.72	0.65	0.78	0.85
DistMult	0.76	0.69	0.82	0.88

DistMult achieved slightly higher performance across all metrics, particularly in tail prediction, indicating its suitability for capturing symmetric relations in our artwork domain knowledge graph.

6.2 Size-Sensitivity Analysis

To evaluate the impact of graph size on embedding model performance, we conducted experiments on three dataset sizes: 20k random triples, 50k random triples, and the full knowledge graph ($\approx 80k$ triples). The TransE model was used for consistency, with the same hyperparameters as before. The results are shown below:

Dataset Size	MRR	Hits@10
20k Triples	0.61	0.73
50k Triples	0.68	0.81
Full KG ($\approx 80k$ Triples)	0.72	0.85

Results confirm that model performance (MRR and Hits@10) improves consistently as the number of triples increases. This is because a larger graph provides more relational information, allowing the embedding model to learn more accurate semantic representations of entities and relations.

6.3 t-SNE Visualization & Nearest-Neighbor Examples

t-SNE (t-distributed Stochastic Neighbor Embedding) was used to project the 100-dimensional TransE embeddings into a 2D space for visualization. Entities were color-coded by their type (Person: blue, Artwork: red, Location: green, Organization: yellow), and the resulting plot showed distinct clusters for each entity type, validating that the model captured meaningful semantic regularities.

Note: Insert the t-SNE visualization screenshot here (a scatter plot with color-coded entity clusters).

Nearest-neighbor queries were also performed to verify the embedding quality. For example, the nearest neighbors of “Leonardo da Vinci” (PERSON) included other painters such as “Michelangelo” and “Raphael”, while the nearest neighbors of “Mona Lisa” (WORK_OF_ART) included other artworks like “The Last Supper” and “Starry Night”. These results demonstrate that the model learned semantically meaningful entity embeddings.

6.4 Entity Filtering and Graph Refinement

In this second phase, additional preprocessing was applied to improve the quality and usability of the extracted entities. The dataset was cleaned by removing null values, noisy entries such as URLs, overly long entity names, and duplicates. Only the most relevant semantic types were retained, namely PERSON, WORK_OF_ART, GPE (mapped to LOCATION), and ORG (mapped to ORGANIZATION).

Initially, the dataset contained 2190 entities. To improve interpretability, a frequency-based filtering step was applied by keeping only the top 20 most frequent entities, resulting in a reduced set of 44 entities. This significantly improves clarity, although it may remove useful information.

6.5 Knowledge Graph Construction

A directed knowledge graph was constructed using NetworkX, where nodes represent entities and edges represent semantic relationships inferred from co-occurrence within the same document. Simple rule-based logic was used to assign relations: for example, a WORK_OF_ART connected to a PERSON is labeled as CREATED_BY, while a WORK_OF_ART connected to a LOCATION is labeled as LOCATED_IN.

However, due to the strong filtering applied earlier, the resulting graph contains only 12 nodes and no edges. This illustrates a trade-off between data quality and relational richness, as excessive filtering can eliminate meaningful connections.

6.6 Graph Visualization

To facilitate interpretation, the graph was visualized using PyVis and exported as an interactive HTML file. Nodes are color-coded according to their type (e.g., PERSON, WORK_OF_ART, LOCATION), and the interface allows users to explore relationships dynamically.

Although the visualization is effective in principle, its usefulness is limited here due to the absence of edges in the graph.

6.7 Document Retrieval (TF-IDF)

In parallel, a simple document retrieval system was implemented using TF-IDF vectorization. Cleaned documents were loaded from the preprocessing step and transformed into vector representations. Cosine similarity was then used to retrieve the most relevant documents given a query.

The dataset includes four documents, which is sufficient for demonstrating the pipeline but remains limited in terms of coverage and retrieval performance.

6.8 Hybrid Retrieval (KG-RAG)

A hybrid retrieval approach was developed by combining text-based retrieval with knowledge graph enrichment. Given a query, the system first retrieves relevant documents using TF-IDF, then attempts to identify entities mentioned in the query and retrieve their neighbors in the graph.

This approach is designed to enhance answers by integrating both contextual text and structured knowledge. However, due to the lack of edges in the graph, the contribution of the knowledge graph remains minimal in this implementation.

6.9 Question Answering System

A lightweight question-answering system was implemented to demonstrate the KG-RAG concept. The system combines retrieved document context with potential knowledge graph information and applies simple rule-based logic to answer specific questions, such as identifying the creator or location of an artwork.

In practice, while the system successfully retrieves relevant textual context, it fails to leverage the knowledge graph effectively. As a result, answers remain generic and rely primarily on the document content rather than structured knowledge.

6.10 Limitations

This phase highlights several limitations. The aggressive filtering strategy significantly reduced the size of the graph and removed most relationships, resulting in a disconnected structure. Consequently, the knowledge graph cannot effectively contribute to reasoning or answer generation.

Future improvements could include relaxing filtering constraints, incorporating more advanced relation extraction methods, and expanding the dataset to improve both graph completeness and retrieval performance.

7. Critical Reflection

7.1 KB Quality Impact

The quality of the knowledge base directly impacts all subsequent tasks. The initial RDF graph (td4_kb_final.ttl) with 100+ triples and 200+ entities provided a solid foundation for KGE and RAG, while the filtered graph (12 nodes, no edges) failed to support effective reasoning and hybrid retrieval. This underscores the importance of balancing data cleaning with graph richness—excessive filtering undermines the KB’s utility, while insufficient cleaning introduces noise that degrades model performance.

7.2 Noise Issues

Noise in the raw web data (HTML tags, advertisements, irrelevant content) was effectively addressed by the cleaning pipeline (`clean_data.py`), but residual noise in entity extraction (e.g., ambiguous entities, incorrect type labeling) still affected KB quality. For example, type ambiguity in words like “Academy” led to inconsistent entity categorization, which in turn impacted relation extraction and embedding learning. Future work could incorporate more advanced NER models (e.g., BERT-based) to reduce such noise.

7.3 Rule-based vs Embedding-based Reasoning

Rule-based reasoning (e.g., SWRL) and embedding-based reasoning (e.g., TransE, DistMult) each have distinct strengths and limitations. Rule-based reasoning is highly interpretable—SWRL rules explicitly define logical relationships, making it easy to validate and debug inferred knowledge. However, it is brittle to noise and incomplete data; if the KB lacks required triples, the rules cannot infer new knowledge. Embedding-based reasoning, by contrast, is robust to incompleteness and can capture implicit patterns in the data, supporting tasks like entity similarity and link prediction. However, it lacks transparency—model decisions are difficult to interpret, and errors in embeddings can lead to incorrect inferences. In our project, SWRL provided clear inference paths for explicit relations, while KGE enabled better generalization for entity similarity. A hybrid approach that combines rule-based logic to guide embedding learning would likely yield better results.

7.4 Improvements for Future Work

Several improvements could enhance the project’s performance and completeness: (1) Relax entity filtering constraints to retain more meaningful relationships, avoiding disconnected graphs; (2) Use advanced relation extraction models (e.g., REBEL) to improve the quality and quantity of relational triples; (3) Integrate external knowledge graphs (e.g., Wikidata) for KB expansion, adding more attributes and relations; (4) Develop a more robust NL-to-SPARQL translation model (e.g., fine-tuning a language model) to improve RAG accuracy; (5) Implement a hybrid reasoning system that combines SWRL rules and KGE to leverage the strengths of both paradigms.

Git hub link : <https://github.com/Elisee-F/Projet-Webdatamining>